AA411

Diagnoser Test Procedure

Team Name: ROFASO

Project Name: Damage Assessment Protocol (D.A.P.)

Function Name: Diagnoser

Written By: Oleksiy Ployakov and Mak Sukimoto

Date: 5/12/2026

Test Dates: 05/17/2026

Test Team:

Name	Initials
Mak Sukimoto	MS
Josh Rolfe	JR
Holland Kantner	HK
Oleksiy Polyakov	OP
Dante Weerasooriya	DW

Contents

1 Objective of Test Procedure

The goal of this procedure is to test that full function of the Diagnoser can input data, compare the data to ensure that there is a fault case, the isolate the fault case and present the potential matches in a table and present a best fit fault case.

The following are the requirements that will be tested using this procedure.

- **FDR-1:** The Diagnoser shall diagnose singular wing or rudder faults between 0-100% flight surface span remaining.

2 Measurements

Variable	Description	Units	Measurement Method
MAPE	mean absolute percent error	All	Percent Difference

3 Equipment Required

Equipment/Tools	Description	Qty	Check
Laptop	At least 8 GB RAM and 256 GB SSD is required	1	
MATLAB R2024a	Simulation environment. Add ons needed: • Aerospace Toolbox • Control System Toolbox • Optimization Toolbox • Robust Control Toolbox	1	
Test Cases in a Matrix	Input Dataset	1	
Fault Comparator	Compares Cases	1	
Fault Isolator Code outputs	outputs of percentage difference	1	
Seaglider Data	Mission Data	1	

4 Test Procedure

i Test Case ID: `TC-FDR-01` — Diagnoser Validation

1. **Environment Initialization**
   (a) Open MATLAB
   (b) Ensure the working directory contains `D_fault_isolator.m`

   `missioncompare_version_4`, `missioncompare_version_holland_4.m`, and the required `.eng` data files.

2. **Data Ingestion and Pre-processing**
   Execute the following example commands to load the mission data and digital twin library:

```
%% Dive 6
MI_Data = missioncompare_unpack_eng_to_dive_data('p1940006.eng');

DT_Lib.Nominal = missioncompare_unpack_eng_to_dive_data('p1940005.eng');
DT_Lib.Rudder  = missioncompare_unpack_eng_to_dive_data('p1940008.eng');
DT_Lib.Wing    = missioncompare_unpack_eng_to_dive_data('p1940011.eng');
```

Listing 1: Data Ingestion Script

3. **Comparator Execution**
   Run the comparator version 4 to generate the diagnostic structures for each library case:

```
fprintf('Executing Tier 1: Windowed Fault Detection...\n');
detection_result = missioncompare_version_4(DT_Lib.Nominal, MI_Data);

out_N = detection_result; % Already ran this
out_R = missioncompare_version_4(DT_Lib.Rudder, MI_Data);
out_W = missioncompare_version_4(DT_Lib.Wing, MI_Data);
```

Listing 2: Comparator Execution

4. **Fault Isolator Ranking**
   Execute the isolator to perform the final tabulation and ranking:

```
[RankingTable, BestMatch] = D_fault_isolator(out_N, out_R, out_W);
```

Listing 3: Isolator Execution

5. **Results Verification**
   Inspect the generated `RankedTable` and verify the following criteria:

Verification Point	Expected Result	Pass/Fail
Table Generation	`RankedTable` exists in workspace as a 2-column table.	
Ranking Order	`Percent_Match_Error` is sorted in ascending order (lowest percent difference to highest percent difference)	
Diagnosis Accuracy	`BestMatch` matches the known fault case.	

6. **Data Collection**
   Save data and upload to Oceans Google Drive.

   (a) AA Resilient Ocean Flight Autonomy for Seaglider Operation → Test Results → Diagnoser

Shut Down

1. Save `Diagnoser_Draft.m`

   (a) Saved? Yes/No ________________

2. Close MatLab

   (a) MatLab Closed? Yes/No ________________

3. Shut down laptop

   (a) Power Off? Yes/No ________________